



Structures

By: Mir Faraz Ali
Programming Fundamentals



What is a Structure?

- Structure is a collection of variables of different data types under a single name.
- For example: You want to store some information about a person: his/her name, citizenship number and salary. You can easily create different variables name, citNo, salary to store these information separately.

C++ Structures:

- If, in the future, you would want to store information about multiple persons, what you will do ?
- you'd need to create different variables for each information per person: name1, citNo1, salary1, name2, citNo2, salary2
- You can easily visualize how big and messy the code would look. Also, since no relation between the variables (information) would exist, it's going to be a scary task.

C++ Structures:

- A better approach will be to have a collection of all related information under a single name Person, and use it for every person. Now, the code looks much cleaner, readable and efficient as well.
- This collection of all related information under a single name Person is a structure.

How to declare a structure in C++ programming?

- The struct keyword defines a structure type followed by an identifier (name of the structure).
- Then inside the curly braces, you can declare one or more members (declare variables inside curly braces) of that structure. For example:
- Here a structure person is defined which has three members: name, age and salary.
- When a structure is created, no memory is allocated.
- You can imagine it as a datatype.

How to define a structure variable?

- Once you declare a structure person as above. You can define a structure variable as:
- Person ahmed;
- Here, a structure variable ahmed is defined which is of type structure Person.
- When structure variable is defined, only then the required memory is allocated by the compiler.
- the memory of float is 4 bytes, memory of int is 4 bytes and memory of string is 8 byte. Hence, 16 bytes of memory is allocated for structure variable ahmed.

How to access members of a structure?

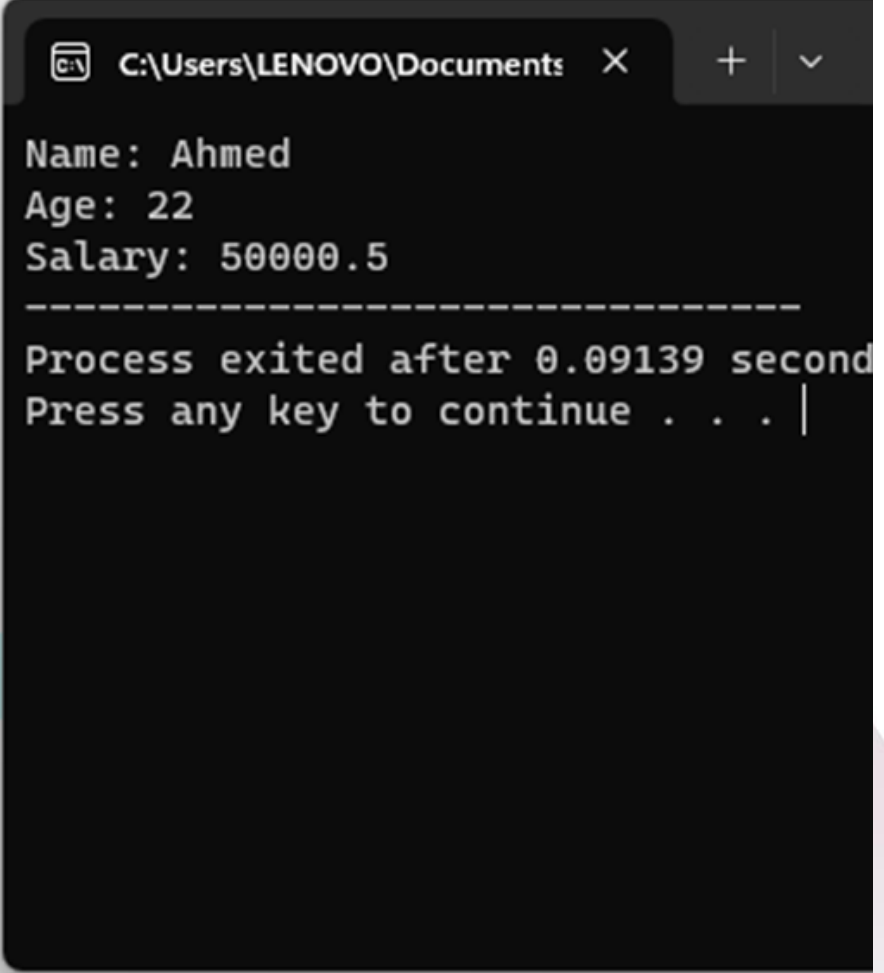
- The members of structure variable is accessed using a dot (.) operator.
- Suppose, you want to access age of structure variable ahmed and assign it 22 to it. You can perform this task by using following code below:
- `ahmed.age=22;`

Example: C++ Structure

- Named Structures:

By giving a name to the structure, you can treat it as a data type. This means that you can create variables with this structure anywhere in the program at any time: like example in this slide

```
structure.cpp
1  #include <iostream>
2  using namespace std;
3
4  struct Person
5  {
6      string name;
7      int age;
8      float salary;
9  };
10 int main()
11 {
12     Person p1;
13     p1.name = "Ahmed";
14     p1.age = 22;
15     p1.salary = 50000.50;
16     cout << "Name: " << p1.name << endl;
17     cout << "Age: " << p1.age << endl;
18     cout << "Salary: " << p1.salary;
19     return 0;
20 }
```



```
C:\Users\LENOVO\Documents
Name: Ahmed
Age: 22
Salary: 50000.5
-----
Process exited after 0.09139 second
Press any key to continue . . .
```


Example: C++ Structure

- unnamed Structures:

An unnamed structure, also known as an anonymous structure, doesn't have a specific name associated with it.

```
structure.cpp
1  #include <iostream>
2  using namespace std;
3
4
5  int main(){
6      struct{
7          string name;
8          int age;
9          float salary;} p1;
10
11      p1.name = "Ahmed";
12      p1.age = 23;
13      p1.salary = 65000.50;
14      cout << "Name: " << p1.name << endl;
15      cout << "Age: " << p1.age << endl;
16      cout << "Salary: " << p1.salary;
17      return 0;
18  }
```

```
C:\Users\LENOVO\Documents
Name: Ahmed
Age: 23
Salary: 65000.5
-----
```

```
Process exited after 0.08141 seconds
Press any key to continue . . .
```

Structure of Diverse Data:

“Diverse data” means different types of information grouped inside a structure.

A structure can store:

- int
- float
- char
- string
- double
- bool

All inside one structure.

Example:

```
struct Employee {  
    int id;  
    string name;  
    float salary;  
    char grade;  
};
```

- id $\rightarrow$ integer
- name $\rightarrow$ string
- salary $\rightarrow$ float
- grade $\rightarrow$ character

Structures as Function Arguments:

We can send a structure to a function just like a simple variable.

There are two ways:

A) Pass Structure by Value

- A copy of structure is passed
- Original structure does NOT change

Example:

```
void display(Student s) {  
    cout << s.name;  
}
```

CALL:

- display(st1);

Structures as Function Arguments:

B) Pass Structure by Reference

- Address of structure is passed
- Original structure can be modified

Example:

```
void updateMarks(Student &s) {  
    s.marks = 90;  
}
```

CALL:

```
updateMarks(st1);
```




End Of Class

